Size-Independent Neural Dynamic Programming for Euclidean TSP with $(1+\varepsilon)$-Approximation Guarantee

Anonymous Authors[1]

Abstract

The Traveling Salesman Problem (TSP) has numerous applications in logistics, manufacturing, and circuit design. In this paper, we study the Euclidean TSP. The dynamic programming (DP) methods achieve a $(1+\varepsilon)$-approximation but are often impractical due to their large constant factors, whereas learning-based solvers are fast but heuristic and often sensitive to size extrapolation. To bridge this gap, we propose a size-independent neural DP approach that preserves a $(1+\varepsilon)$-approximation guarantee for the Euclidean TSP. The instance size affects only the number of reuses of our module, enabling cross-size generalization. Building on this foundation, we introduce practical accelerations and provide a modular component that can be integrated into strong heuristic solvers (e.g., Lin–Kernighan–Helsgaun, LKH) with minimal modification. Empirically, our models trained only on instances of size $n = 50$ generalize to TSPLIB instances up to $n = 85900$. When integrated into LKH-3, our hybrid solver achieves about $25\times$–$270\times$ speedup compared to LKH-3 alone, and is competitive with or improves upon existing neural TSP baselines in our evaluation.

1. Introduction

The Euclidean Traveling Salesman Problem (TSP) is a central testbed in geometric optimization and combinatorial algorithm design. Beyond its theoretical role, Euclidean instances arise in practical settings such as transportation/logistics and geometric tool-path planning, and they constitute the most commonly studied TSP family in empirical research. They also underpin standard public benchmarks (e.g., TSPLIB (Reinelt, 1991)) and large-scale challenges (e.g., the World TSP instances (Cook)), making Euclidean TSP a particularly consequential target for methods that aim to be both principled and practical. Formally, given a point set $P \subset \mathbb{R}^d$ with $n := |P|$, we seek a tour T visiting every point that minimizes its length $\mathrm{len}(T)$; denote the optimum value by OPT.

From the theory side, for any $\varepsilon \in (0,1)$ and fixed dimension, polynomial-time approximation schemes (PTAS) return a $(1+\varepsilon)$-approximate tour of Euclidean TSP (Arora, 1998; Mitchell, 1996; Rao & Smith, 1998). Yet decades after these guarantees were established, state-of-the-art practice is still dominated by highly engineered heuristics like Lin–Kernighan style local search and its modern implementations (e.g., LKH-3) that deliver excellent tours at scale but without PTAS-style end-to-end guarantees (Lin & Kernighan, 1973; Helsgaun, 2000; 2017; Johnson & McGeoch, 1997; Applegate et al., 2011). Bridging this theory–practice gap for Euclidean TSP is the starting point of this paper.

A central reason why PTAS frameworks are rarely used in practice is the large hidden constant in their dynamic programming (DP) routines (Arora, 1998; Har-Peled, 2011; Applegate et al., 2011). These algorithms solve the instance via dynamic programming over a hierarchical spatial decomposition of the plane, typically implemented as a *quadtree*: starting from an axis-aligned square that contains all input points, each region (a box) is recursively subdivided into four smaller boxes until a prescribed scale is reached (Figure 1). The DP is computed bottom-up on this quadtree: first solving the smallest boxes and then aggregating their solutions to obtain the solution for larger boxes. The expensive part is not solving each box, but merging boxes: the parent solution must consider many possible ways of combining the child solutions. This merge step is inherently combinatorial and often becomes the practical bottleneck even when the theoretical running time is near-linear for fixed ε (Rao & Smith, 1998).

From the practical side, the strongest TSP solvers remain overwhelmingly heuristic. Over decades, the community has developed highly effective pipelines centered on local search and neighborhood engineering—ranging from classic k-opt moves (e.g., 2-opt/3-opt) to Lin–Kernighan–style

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

For ICML 2026 reviewers: When reviewing this paper, please follow the policy for the use of LLMs as specified on your Reviewer Console. Note that the assigned LLM policy might differ from the policy you indicated during reviewer registration.

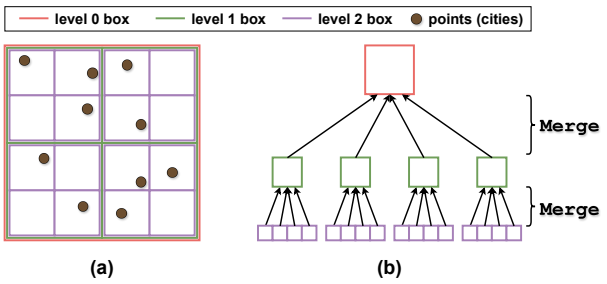


Figure 1. Quadtree dissection and bottom-up DP for Arora/Rao-style Euclidean TSP PTAS. (a) A recursive quadtree partitions the point set. (b) The DP proceeds bottom-up by merging four child cost tables into the parent; this combinatorial merge is the main bottleneck addressed by our size-independent learned merge module.

variable-depth search—further strengthened by candidate sets, fast move evaluation, and carefully tuned search control. Modern implementations such as LKH-3 exemplify this paradigm, routinely producing near-optimal tours on large Euclidean instances at impressive speeds (Helsgaun, 2000; 2017). However, these methods do not provide PTAS-style end-to-end guarantees and can be sensitive to hyperparameters and hand-crafted components (Johnson & McGeoch, 1997; Applegate et al., 2011). This motivates learned guidance that can be inserted into mature pipelines to improve decisions (e.g., candidate selection and move prioritization) without compromising their robustness, while preserving formal structure when possible.

In parallel, learning-based methods for TSP have achieved impressive empirical performance, ranging from end-to-end neural solvers (Kwon et al., 2020; Sun & Yang, 2023; Li et al., 2023; 2024) to hybrid pipelines that predict structural hints (e.g., edge scores or candidate sets) to guide downstream heuristics (Xin et al., 2021; Hudson et al., 2021; Min et al., 2023). However, these approaches are typically heuristic in nature: they do not provide PTAS-style end-to-end guarantees (Bengio et al., 2021). For Euclidean TSP, many learned solvers are sensitive to distribution and size extrapolation (Fu et al., 2021). Models may overfit to training priors, so their edge preferences and decoding can become miscalibrated on new generators or structured benchmarks. As n grows, constructive errors accumulate, refinement budgets may become too small, and candidate predictors trained on small instances can miss rare but critical edges, limiting downstream local search.

Based on these phenomena, a key observation behind our approach is that we do not need to learn a full TSP solver to gain practical speedups with theoretical guarantees. In PTAS-style Euclidean TSP, most of the hidden constant comes from the DP merge routine, while the rest of the backbone primarily provides a certified way to organize and combine local computations. This suggests a principled division of labor: keep the PTAS backbone for end-to-end certification, and learn only the merge primitive to accelerate it. However, this is not merely a drop-in replacement. The merge primitive is a tightly constrained combinatorial operator, applied repeatedly across the quadtree hierarchy (about $O(\log n)$ times), so naïve learned merges can introduce inconsistencies or bias that propagate upward. Our method therefore targets a merge module that is lightweight and composable across levels. Moreover, under Rao's spanner/patching framework (Rao & Smith, 1998), what the merge needs to "know" depends only on ε rather than on the instance size n, enabling a *size-independent* module reusable across scales, and therefore naturally supporting strong size generalization. Our main contributions are as follows:

- **A $(1+\varepsilon)$-approximate PTAS solver with a size-independent neural merge primitive.** We retain an Arora/Rao-style PTAS backbone and learn only the DP *merge* bottleneck via a size-independent neural module. We formalize an entrywise δ-accuracy condition on normalized DP costs and show how it lifts to an end-to-end $(1+\varepsilon)$ approximation guarantee.
- **Plug-and-play acceleration for strong heuristics.** Using the same merge module, we build a drop-in guidance mechanism for LKH-style local search heuristics to accelerate practical solving. Experiments show substantial speedups while maintaining solution quality close to running the heuristic alone. Our module also achieves strong size generalization when training on small instances and evaluating on much larger ones.

Other related works and the organization Related mixed neural–algorithmic efforts have also begun to learn subroutines inside classical PTAS paradigms, including Arora-style quadtree frameworks for other geometric problems (e.g., NN-Steiner for rectilinear Steiner minimum trees (Kahng et al., 2024)) and Baker-paradigm schemes for geometric intersection-graph problems (e.g., NN-Baker for MIS/vertex cover (McCarty et al., 2021)). A broader discussion of geometric PTAS backbones, neural-accelerated dynamic programs, and modern neural/hybrid TSP solvers is deferred to Appendix B due to the space limit. We next outline the organization of the paper: Section 2 reviews the Arora/Rao backbone and the induced DP objects. Section 3 presents our learned merge primitive with verifiable DP semantics and the resulting solver. Section 4 reports the empirical results.

2. Preliminaries

We review Rao's PTAS backbone for Euclidean TSP and introduce the dynamic-programming notation used throughout (Rao & Smith, 1998). The backbone uses a randomized

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

quadtree dissection and a patched sparse graph to restrict attention to a DP-enumerable family of tours that admits a $(1+\varepsilon)$ approximation guarantee. The resulting bottom-up DP repeatedly applies a local merge operator that aggregates the four child subproblems into the parent. For simplicity and clarity, we restrict attention to the two-dimensional Euclidean TSP in what follows; all of our discussions extend to any fixed dimension d.

2.1. Rao-style PTAS backbone and DP notation

Quadtree dissection. Let $P \subset \mathbb{R}^2$ be a set of n points. Following the PTAS framework, we assume (after standard scaling/rounding) that P lies in an axis-aligned bounding square B_{root} whose corners have integer coordinates; let $L := \ell(B_{\text{root}})$.

We construct a rooted quadtree dissection of the bounding square B_{root} as follows (as shown in Figure 1). At level $j \geq 0$, we partition B_{root} by an axis-aligned grid with spacing $L/2^j$, yielding a hierarchical decomposition into squares (boxes) where each box at level j has side length $L/2^j$. To randomize the dissection, we apply a *random shift*: we sample $u = (u_x, u_y)$ uniformly from $[0, L)^2$ and offset all vertical and horizontal grid lines by u_x and u_y, respectively (equivalently, we translate the coordinate system by u), then restrict the resulting partition to B_{root}. We denote the resulting rooted quadtree by $\mathcal{T}$.

Each node of $\mathcal{T}$ corresponds to a square (a box) B. Let $\text{level}(B)$ denote its depth in $\mathcal{T}$ (with the root at level 0), and let $\ell(B)$ denote its side length (so $\ell(B) = L/2^{\text{level}(B)}$). Each internal box B has four children (B_1, B_2, B_3, B_4) obtained by bisecting B horizontally and vertically, hence $\ell(B_i) = \ell(B)/2$ for each child. We call a box B *nonempty* if $P \cap B \neq \emptyset$; the DP and our later bounds only range over nonempty boxes.

Preprocessing: spanner construction and the r-light restriction. Rao's framework constructs a sparse spanner S that contains a near-optimal tour, and then performs Arora-style patching aligned with the dissection to obtain a modified graph S' whose interaction with each box boundary is bounded. Fix $\varepsilon \in (0,1)$, and let constant $r = r(\varepsilon)$ denote the **crossing bound**: for every box $B \in \mathcal{T}$, each side of the boundary ∂B is intersected by S' at most r times (equivalently, S' is an r**-light spanner** with respect to $\mathcal{T}$). We view the dynamic program as optimizing over the restricted (legal) solution family $\mathcal{S}_{legal}$ induced by $(\mathcal{T}, S', r)$.

Let OPT denote the length of an optimal Euclidean TSP tour on P, and let $\text{OPT}(\mathcal{S}_{legal})$ denote the minimum tour length within $\mathcal{S}_{legal}$. The structural results of Arora/Rao ensure $\text{OPT}(\mathcal{S}_{legal}) \leq (1+\varepsilon)\,\text{OPT}$ for an appropriate choice of constant r. The following theorem shows that DP on the r-light spanner S' implies a global approximation guarantee .

Theorem 2.1. *(Rao & Smith, 1998) Fix $\varepsilon \in (0,1)$ and assume $\text{OPT}(\mathcal{S}_{legal}) \leq (1+\varepsilon)\,\text{OPT}$. For any $\varepsilon_{\text{DP}} \in (0,1)$, if a dynamic program returns a tour $T_{\text{DP}} \in \mathcal{S}_{legal}$ with $\text{len}(T_{\text{DP}}) \leq (1+\varepsilon_{\text{DP}})\,\text{OPT}(\mathcal{S}_{legal})$, then $\text{len}(T_{\text{DP}}) \leq (1+\varepsilon_{\text{DP}})(1+\varepsilon)\,\text{OPT}$. In particular, an exact DP over $\mathcal{S}_{legal}$ yields a $(1+\varepsilon)$-approximation.*

Dynamic programming and the merge primitive. After preprocessing, we have a quadtree $\mathcal{T}$ together with an r-light graph S'. By Theorem 2.1, it suffices to run an exact bottom-up DP on $\mathcal{T}$ to obtain a global PTAS. We now specify the DP objects, emphasizing the merge primitive induced by the quadtree recursion.

For each box B, let $\Omega(B)$ denote the set of *boundary states*. A boundary state $\sigma \in \Omega(B)$ specifies (i) which boundary crossing locations on ∂B are active (i.e., which edges of the solution cross ∂B), and (ii) the connectivity among active crossings (e.g., a planar non-crossing pairing of the active crossing locations) restricted by the family $\mathcal{S}_{legal}$. Let $C^\star_B(\sigma)$ be the optimal *cost* of state σ of B: the minimum total length of a partial solution inside B that realizes σ and covers all points of P lying in B (as required by $\mathcal{S}_{legal}$). We define $C^\star_B = \{C^\star_B(\sigma) \mid \sigma \in \Omega(B)\}$ for the optimal *cost table* of B, and define the normalized cost of state σ:

$$\bar{C}^\star_B(\sigma) := \frac{C^\star_B(\sigma)}{\ell(B)}.$$

The DP bottom-up recursion repeatedly invokes a **merge** primitive that combines the four child cost tables to produce the parent cost table; leaf cost tables can be filled naively. Consider an internal box B with children (B_1, B_2, B_3, B_4). Given a parent state $\sigma \in \Omega(B)$, let $\mathcal{F}_B(\sigma)$ be the set of feasible child-state 4-tuples

$$\tau(\sigma) = (\sigma_{B_1}, \sigma_{B_2}, \sigma_{B_3}, \sigma_{B_4}) \tag{1}$$
$$\in \Omega(B_1) \times \Omega(B_2) \times \Omega(B_3) \times \Omega(B_4) \tag{2}$$

that are consistent with σ, meaning (i) child boundary states agree on crossings along shared boundaries, and (ii) the induced connectivity across the four children is compatible with σ. For the TSP under this framework, the merge objective is additive (any patching overhead is absorbed into preprocessing), so

$$C^\star_B(\sigma) = \min_{\tau(\sigma) \in \mathcal{F}_B(\sigma)} \sum_{i=1}^{4} C^\star_{B_i}(\sigma_{B_i}). \tag{3}$$

In normalized form, using $\ell(B_i) = \ell(B)/2$, we equivalently have

$$\bar{C}^\star_B(\sigma) = \min_{\tau(\sigma) \in \mathcal{F}_B(\sigma)} \frac{1}{2} \sum_{i=1}^{4} \bar{C}^\star_{B_i}(\sigma_{B_i}). \tag{4}$$

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

State-space size and size-independent interface of the merge primitive. Under the r-light restriction, each side of B is crossed at most r times, hence ∂B has at most $4r$ crossings in total. A state $\sigma \in \Omega(B)$ is determined by (i) a subset of these crossings and (ii) a planar non-crossing pairing/connectivity on the active crossings. Therefore, while each *state description* involves only $O(r)$ crossings, the total number of possible states is bounded by $r^{O(r)}$, depending only on r (and thus only on ε) and not on n.

Lemma 2.2 (State-space bound)**.** *For every box $B \in \mathcal{T}$, the number of boundary states satisfies $|\Omega(B)| \leq r^{O(r)}$.*

3. Our method

In this section, we formalize the DP merge primitive and introduce our learned merge primitive. We then prove that our method yields a global $(1+\varepsilon)$-approximation, and finally describe a plug-and-play acceleration module that complements strong heuristics.

As discussed in Section 1, the main obstacle to making PTAS-style approaches practical (and scalable) is the merge step in the dynamic program. For an internal box B with children (B_1, B_2, B_3, B_4), each child carries a cost table $C_{B_i} : \Omega(B_i) \to \mathbb{R}$ that stores the best local objective value for every discrete boundary state.

First, we define the **normalized geometric feature** $Geom(B)$: For any box B, $Geom(B)$ as the list of coordinates of the intersection points where edges of S' cross the boundary ∂B, normalized by $\ell(B)$ and ordered along ∂B (so the length of $Geom(B)$ is bounded by $O(r)$). The merge primitive takes as input the four child tables $\{C_{B_i}\}_{i=1}^4$ together with normalized geometric features $\{Geom(B), Geom(B_1), \ldots, Geom(B_4)\}$. Then the merge primitive outputs (i) the parent table $C_B : \Omega(B) \to \mathbb{R}$ and (ii) for each $\sigma \in \Omega(B)$ an argmin witness (backpointer) $\tau(\sigma) = (\sigma_{B_1}, \sigma_{B_2}, \sigma_{B_3}, \sigma_{B_4})$

3.1. Our learned merge primitive

We approximate the merge with a learned procedure that decodes a continuous child-state tuple and then deterministically PARSEs it into a discrete child-state tuple, while keeping feasibility and cost evaluation exact. Throughout, we operate on normalized costs $\bar{C}_B(\sigma) := C_B(\sigma)/\ell(B)$ so that the representation does not depend on the absolute box scale.

Intuition. Let $\tilde{\sigma}$ denote a *continuous* boundary state vector, viewed as a relaxation of a discrete state: its entries are scores aligned with the ordered crossing tokens in $Geom(B)$ (with masked padding). Our encoder-decoder structure predicts continuous child boundary states, and a deterministic parser PARSE rounds them back to discrete states in $\Omega(B_i)$. We then verify feasibility and compute the merge value exactly using the DP tables, so learning is responsible only for reducing expensive enumeration, not for enforcing constraints.

Given $(Geom(B), \{Geom(B_i)\}_{i=1}^4, \{\bar{C}_{B_i}\}_{i=1}^4)$, our merge runs:

1. **Encode children.** An encoder produces embeddings h_{B_i} from each $(Geom(B_i), \bar{C}_{B_i})$.
2. **Decode continuous child boundary states.** For each parent state $\sigma \in \Omega(B)$, a conditional decoder takes $(\sigma, Geom(B), h_{B_1}, \ldots, h_{B_4})$ and outputs a continuous child-state 4-tuple
$$\tilde{\tau}(\sigma) = (\tilde{\sigma}_{B_1}, \tilde{\sigma}_{B_2}, \tilde{\sigma}_{B_3}, \tilde{\sigma}_{B_4}),$$
where each $\tilde{\sigma}_{B_i}$ is a fixed-dimensional continuous boundary state for child B_i, whose entries can be interpreted as scores (or probabilities after a sigmoid) indicating how likely each local boundary crossing is selected.
3. **Parse, verify, and evaluate.** A deterministic procedure PARSE maps $\tilde{\tau}(\sigma)$ to a discrete child-state tuple $\tau(\sigma) = (\sigma_{B_1}, \sigma_{B_2}, \sigma_{B_3}, \sigma_{B_4})$. We then (i) check whether $\tau(\sigma) \in \mathcal{F}_B(\sigma)$, and (ii) compute the merge value *exactly* from the child tables, storing $\tau(\sigma)$ as the backpointer. If $\tau(\sigma) \notin \mathcal{F}_B(\sigma)$, we trigger an entry-level fallback (exact enumeration for this σ only).

Child encoder. The encoder is invoked once per child in each merge and serves as a *learned summary* of the local DP subproblem. Concretely, it maps $(Geom(B_i), \bar{C}_{B_i})$ to a fixed-dimensional embedding h_{B_i}. We implement it as a lightweight multi-head attention module over box scale-normalized geometry tokens from $Geom(B_i)$, augmented with a compact summary of the normalized table $\bar{C}_{B_i}$. This design is permutation-invariant to token ordering and strictly local to each child, matching the quadtree recursion. Moreover, by using normalized geometric features $Geom(\cdot)$ (relative to the box scale), the same encoder parameters can be shared across all quadtree levels, which is essential for size generalization.

Boundary-state decoder. A seemingly natural choice is to learn a direct mapping from $(\sigma, Geom(B), h_{B_1}, \ldots, h_{B_4})$ to a discrete child-state tuple in $\Omega(B_1) \times \cdots \times \Omega(B_4)$. However, this output space is large and tightly coupled across children: small prediction errors can easily yield an infeasible tuple, and a single wrong discrete decision is hard to repair downstream.

Instead, we decode a continuous relaxation. For each parent state $\sigma \in \Omega(B)$, the decoder outputs a continuous child-

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

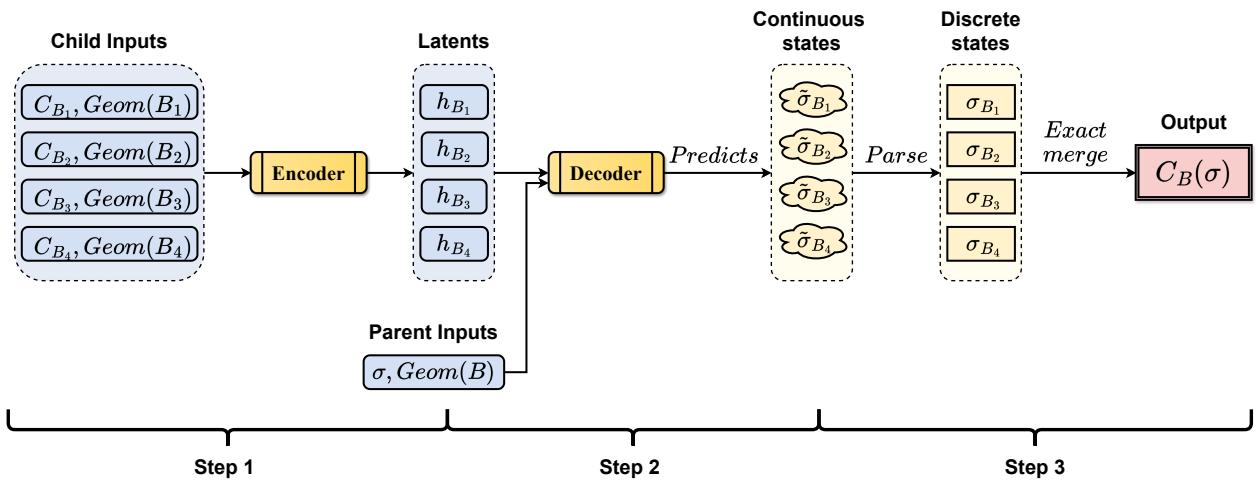


Figure 2. **NN-aligned learned merge for computing parent DP entries.** Given normalized child cost tables and geometric features, an encoder produces latent summaries h_{B_i}. For each parent boundary state σ, a decoder predicts continuous child states $\tilde{\tau}(\sigma)$, which are discretized by PARSE to $\tau(\sigma)$. We then check the feasibility of $\tau(\sigma)$ and compute the parent entry $C_B(\sigma)$ (and backpointer) exactly by table lookup/merge; if infeasible, we fall back to exact per-σ enumeration.

state 4-tuple

$$\tilde{\tau}(\sigma) = (\tilde{\sigma}_{B_1}, \tilde{\sigma}_{B_2}, \tilde{\sigma}_{B_3}, \tilde{\sigma}_{B_4}),$$

where each $\tilde{\sigma}_{B_i}$ is a continuous boundary state for child B_i, whose entries can be interpreted as scores indicating how likely each local boundary crossing is selected. We implement the decoder as a lightweight attention module: it embeds σ as a query, conditions on $Geom(B)$, and attends to $(h_{B_1}, \ldots, h_{B_4})$ to produce the four child states in parallel. Crucially, we do not use $\tilde{\sigma}$ to define the objective; it only guides a deterministic rounding step. Combined with PARSE and exact DP scoring, this yields a stable approximation that preserves verifiability.

Parser, feasibility check, and exact evaluation. The PARSE step projects continuous child states back to the discrete DP state space. Concretely, given $\tilde{\tau}(\sigma)$, it returns a discrete tuple $\tau(\sigma) = (\sigma_{B_1}, \sigma_{B_2}, \sigma_{B_3}, \sigma_{B_4})$. We then enforce DP semantics explicitly: we accept only if $\tau(\sigma) \in \mathcal{F}_B(\sigma)$ and compute the merge value exactly from the child tables. Therefore, learning is confined to predicting a continuous relaxation (via $\tilde{\tau}$), while PARSE deterministically rounds it to a discrete tuple; feasibility and costs remain governed by the original DP rules. The entry-level fallback guarantees correctness: whenever PARSE fails to produce a feasible tuple for (B, σ), we revert to exact enumeration for that single entry. Optionally, PARSE can emit a small top-K list of discrete tuples to reduce fallback frequency. A concrete deterministic instantiation of PARSE (including feasibility checks and a constant-size decoding DP) is provided in Appendix A.1.

In the next subsection, we formalize a local accuracy condition for each learned merge call and show how it lifts to a global approximation guarantee for the full DP.

3.2. Theoretical guarantees

We define an entrywise δ-accuracy notion for each learned merge call on normalized costs and show that these local additive errors accumulate in a controlled way along the quadtree. This yields an explicit bound at the root and, together with the PTAS backbone, implies an end-to-end approximation guarantee.

For an internal box $B \in \mathcal{T}$ with children $(B_1, \ldots, B_4)$, fix a parent state $\sigma \in \Omega(B)$. Recall that $\mathcal{F}_B(\sigma) \subseteq \Omega(B_1) \times \cdots \times \Omega(B_4)$ is the set of feasible *discrete* child-state 4-tuples consistent with σ. We write a child-state tuple as $\tau(\sigma) = (\sigma_{B_1}, \sigma_{B_2}, \sigma_{B_3}, \sigma_{B_4}) \in \mathcal{F}_B(\sigma)$, where each $\sigma_{B_i} \in \Omega(B_i)$ denotes the child boundary state of B_i.

Let C_B be the cost table produced by our learned merge primitive (with feasibility check / fallback), and let $\bar{C}_B(\sigma) := C_B(\sigma)/\ell(B)$ be its normalized entry. Comparing to the exact normalized merge (4), we call a merge call *δ-accurate* if for every internal box B and every $\sigma \in \Omega(B)$, we have

$$0 \;\le\; \bar{C}_B(\sigma) - \min_{(\sigma_{B_1},\ldots,\sigma_{B_4}) \in \mathcal{F}_B(\sigma)} \Big[\tfrac{1}{2} \sum_{i=1}^{4} \bar{C}_{B_i}(\sigma_{B_i}) \Big] \;\le\; \delta, \tag{5}$$

and in the following proofs we assume the network is trained well enough so that we can take $\delta = \varepsilon/n$; see Remark 3.4 for discussion.

Theorem 3.1 (Learned merge implies a global PTAS)**.** *Assume the backbone preprocessing yields* $\mathrm{OPT}(\mathcal{S}_{legal}) \le (1+\varepsilon)\,\mathrm{OPT}$. *Run the bottom-up DP on the quadtree $\mathcal{T}$ using our learned merge primitive and reconstruct a tour $T \in \mathcal{S}_{legal}$ by traceback. If every internal merge satis-*

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

fies (5)*, then*

$$\mathrm{len}(T) \le (1 + O(\varepsilon))\,\mathrm{OPT}, \tag{6}$$

Moreover, our learned merge is size-independent: its architecture and I/O dimensions depend only on $r(\varepsilon)$ (hence are independent of the instance size n).

We prove Theorem 3.1 via the following two lemmas: the first controls how the local merge errors accumulate within any subtree, and the second upper-bounds the total accumulated error at the root.

Lemma 3.2 (Accumulated error inside a subtree)**.** *Assume every internal merge satisfies* (5)*. For any box B, let $\mathrm{Int}(B)$ be the set of internal boxes in the subtree rooted at B, and define*

$$error(B) := \delta \sum_{B' \in \mathrm{Int}(B)} \ell(B').$$

Then for every box B and every $\sigma \in \Omega(B)$,

$$C_B(\sigma) \;\le\; C_B^\star(\sigma) + error(B). \tag{7}$$

Proof. We propagate the entrywise additive error in (5) through the quadtree recursion.

Unnormalized local error. Multiplying (5) by $\ell(B)$ and using $\ell(B_i) = \ell(B)/2$ gives, for every internal B and every $\sigma \in \Omega(B)$,

$$0 \;\le\; C_B(\sigma) - \min_{(\sigma_{B_1},\dots,\sigma_{B_4}) \in \mathcal{F}_B(\sigma)} \sum_{i=1}^{4} C_{B_i}(\sigma_{B_i}) \;\le\; \delta\,\ell(B). \tag{8}$$

We prove (7) by induction on the subtree height. If B is a leaf, the entry is computed exactly by brute force, so $C_B(\sigma) = C_B^\star(\sigma)$ and $error(B) = 0$. Now let B be internal with children (B_1, B_2, B_3, B_4) and fix $\sigma \in \Omega(B)$. Let

$$\tau^\star(\sigma) = (\sigma^\star_{B_1}, \sigma^\star_{B_2}, \sigma^\star_{B_3}, \sigma^\star_{B_4}) \in \mathcal{F}_B(\sigma)$$

be a minimizer in the exact merge, i.e.,

$$C_B^\star(\sigma) \;=\; \sum_{i=1}^{4} C^\star_{B_i}(\sigma^\star_{B_i}).$$

Evaluating (8) at this tuple gives

$$C_B(\sigma) \;\le\; \sum_{i=1}^{4} C_{B_i}(\sigma^\star_{B_i}) \;+\; \delta\,\ell(B). \tag{9}$$

By the induction hypothesis applied to each child B_i,

$$C_{B_i}(\sigma^\star_{B_i}) \;\le\; C^\star_{B_i}(\sigma^\star_{B_i}) + error(B_i).$$

Summing over $i = 1, \dots, 4$ and plugging into (9) yields

$$\begin{aligned} C_B(\sigma) &\le \sum_{i=1}^{4} C^\star_{B_i}(\sigma^\star_{B_i}) \;+\; \sum_{i=1}^{4} error(B_i) \;+\; \delta\,\ell(B) \\ &= C_B^\star(\sigma) + error(B), \end{aligned}$$

where we used $\mathrm{Int}(B) = \{B\} \cup \mathrm{Int}(B_1) \cup \cdots \cup \mathrm{Int}(B_4)$ and the definition of $error(\cdot)$. □

Lemma 3.3 (Bounding $error(B_{\mathrm{root}})$)**.** *Let $L := \ell(B_{\mathrm{root}})$. Then*

$$\sum_{\substack{B \text{ nonempty} \\ B \in \mathcal{T}}} \ell(B) \;\le\; 2nL, \tag{10}$$

and hence $error(B_{\mathrm{root}}) \le 2n\delta L$. In particular, taking $\delta = \varepsilon/n$ yields $error(B_{\mathrm{root}}) \le 2\varepsilon L$.

Proof. At level j of the quadtree, all boxes have side length $L/2^j$, and there are at most n nonempty boxes (since each nonempty box contains at least one point and each point lies in exactly one box per level). Therefore

$$\sum_{\substack{B \text{ nonempty} \\ \mathrm{level}(B)=j}} \ell(B) \;\le\; n \cdot \frac{L}{2^j}.$$

Summing over levels gives (10). Since $\mathrm{Int}(B_{\mathrm{root}})$ is a subset of nonempty boxes,

$$error(B_{\mathrm{root}}) = \delta \sum_{B' \in \mathrm{Int}(B_{\mathrm{root}})} \ell(B') \;\le\; \delta \sum_{\substack{B \text{ nonempty} \\ B \in \mathcal{T}}} \ell(B) \tag{11}$$

$$\le\; 2n\delta L. \tag{12}$$

Setting $\delta = \varepsilon/n$ yields $error(B_{\mathrm{root}}) \le 2\varepsilon L$. □

Proof of Theorem 3.1. Let B_{root} be the root box and choose $\sigma^\star \in \Omega(B_{\mathrm{root}})$ such that $C^\star_{B_{\mathrm{root}}}(\sigma^\star) = \mathrm{OPT}(\mathcal{S}_{legal})$. Since the algorithm takes a minimum over σ at the root,

$$\begin{aligned} \mathrm{len}(T) &= \min_{\sigma \in \Omega(B_{\mathrm{root}})} C_{B_{\mathrm{root}}}(\sigma) \le C_{B_{\mathrm{root}}}(\sigma^\star) \\ &\le C^\star_{B_{\mathrm{root}}}(\sigma^\star) + error(B_{\mathrm{root}}) \\ &= \mathrm{OPT}(\mathcal{S}_{legal}) + error(B_{\mathrm{root}}), \end{aligned}$$

where we used Lemma 3.2 at the last inequality. If we further take $\delta = \varepsilon/n$, then Lemma 3.3 gives $error(B_{\mathrm{root}}) \le 2\varepsilon L$; combining with $\mathrm{OPT}(\mathcal{S}_{legal}) \le (1+\varepsilon)\,\mathrm{OPT}$ (and the standard preprocessing relation between L and OPT) yields a $(1 + O(\varepsilon))$ bound. We set $\varepsilon' = \varepsilon/C$ (for a fixed constant C) in the backbone so that the overall bound stated as $(1 + O(\varepsilon'))\mathrm{OPT}$ implies a final $(1+\varepsilon)\mathrm{OPT}$ guarantee.

For the size-independent part of Theorem 3.1, the merge network is *size-independent* because it interacts with the DP

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

only via a fixed-size boundary interface. For each box B, the DP maintains a table over boundary states whose cardinality is bounded solely by $r(\varepsilon)$; hence the network embeddings and I/O dimensions depend only on $r(\varepsilon)$ and not on the instance size n. Geometric features are scale-normalized (e.g., via $Geom(B)$), so enlarging n merely yields more quadtree boxes where the same merge module is reused, establishing the size-independent claim in Theorem 3.1.

□

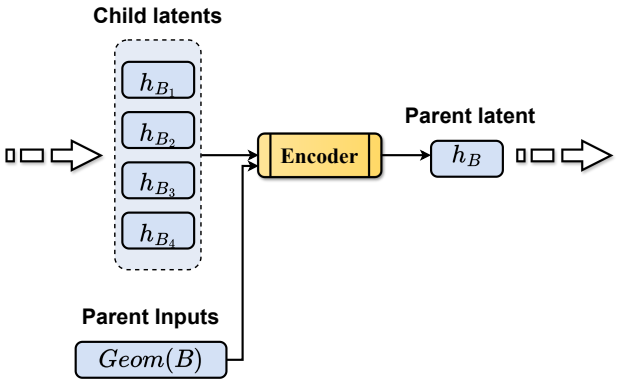
Remark 3.4 (On choosing $\delta = \varepsilon/n$). In the analysis above we instantiate $\delta = \varepsilon/n$, i.e., we assume the merge network is trained well enough so that each internal merge call satisfies (5) with this entrywise normalized error level. This is a sufficient worst-case condition that keeps the cumulative additive error at the root within $O(\varepsilon)$ times the instance scale. Empirically, our experiments indicate that the overall solution error does not grow noticeably with n when applying the same attention-based merge model across problem sizes. A finer-grained guarantee could replace worst-case δ with an expected error that depends on fallback frequency; we leave such refinements to future work.

3.3. Practical Instantiation

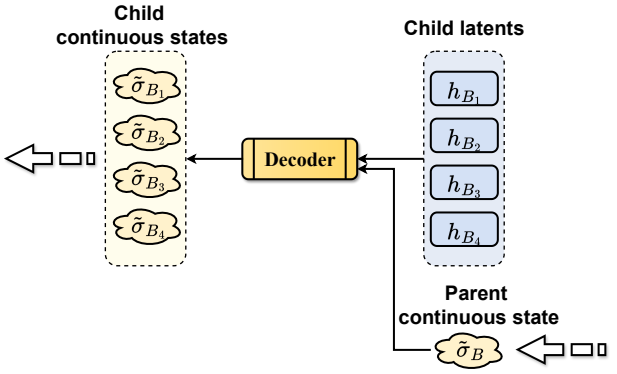
As described above, our continuous-state decoding, followed by deterministic PARSE and exact cost computing, replaces the brute-force search and provides a feasible tuple per σ. In an engineering deployment, we can further accelerate inference by removing the remaining per-σ decoding loop: rather than invoking the decoder up to $|\Omega(B)|$ times per box, we invoke it only once. Inspired by the mixed neural-algorithmic instantiation for *Rectilinear Steiner Minimum Tree (RSMT)* problem in NN-Steiner (Kahng et al., 2024), we reuse the same fixed-size neural interface and change only the execution schedule for practical acceleration. We keep the same encoder–decoder structure and the continuous boundary state representation; the only change is the *scheduling*.

Two-pass schedule. As shown in Figure 3, we change the execution schedule of encoder–decoder structure to a two-pass traversal of the quadtree. In **Step 1** (bottom-up encoding), we compute and cache a latent representation h_B for every box B by aggregating its children latents (with a lightweight leaf encoder at the leaves). In **Step 2** (top-down decoding), we visit each box exactly once under a single continuous boundary state $\tilde{\sigma}_B$ provided by its parent, and one decoder call produces the child states $(\tilde{\sigma}_{B_1}, \tilde{\sigma}_{B_2}, \tilde{\sigma}_{B_3}, \tilde{\sigma}_{B_4})$ that are propagated recursively. This turns the original "$|\Omega(B)|$ times per box" decoding into once per box; full constructions are deferred to Appendix A.4.

Post-processing and training (overview). In practical mode we bypass the exact DP evaluation. For each box, the decoder outputs a continuous boundary state $\tilde{\sigma}_B$, whose



(a) **Bottom-up.** Encode each box by aggregating child latents (and local geometry) into a cached representation h_B.



(b) **Top-down.** Given the parent state $\tilde{\sigma}_B$, decode and propagate child continuous boundary states $\{\tilde{\sigma}_{B_i}\}_{i=1}^4$, whose entries score the likelihood of opening each local boundary crossing.

Figure 3. **Two-pass practical instantiation.** A bottom-up pass caches box latents, and a single top-down pass decodes continuous boundary states for all boxes, providing inexpensive guidance signals for downstream heuristics.

entries can be interpreted as scores (or probabilities after a sigmoid) indicating how likely each local boundary crossing is selected. We aggregate these scores into sparse edge weights on the graph S', and use them as guidance for a downstream heuristic solver to construct a warm-start tour and to bias the candidate edges explored by LKH. Our practical method is a heuristic acceleration without worst-case approximation guarantees. For training, we use high-quality teacher tours on small instances: for each visited box we label which boundary crossings are realized by the teacher, and minimize a masked binary cross-entropy loss under free-running conditioning to match the inference schedule. Details are deferred to Appendix A.4 and Appendix G.

4. Experiments

All experiments are run on a single server equipped with 8× NVIDIA RTX 6000 Ada Generation GPUs and Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz (2×16 cores, 64 threads), 1TB RAM. Reported runtimes are end-to-end wall-clock time measured from data loading (including

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 1. Large-scale TSPLIB evaluation summary. 'Gap' is computed w.r.t. 'LKH-3' (negative means better). Time is wall-clock seconds (**POMO** results are unavailable for most large TSPLIB cases because it fails to finish within a reasonably long time budget).

		Ours+Greedy			Ours+LKH			POMO			NeuroLKH			LKH-3	
Instance	N	Obj	Gap (%)	Time (s)	Obj	Gap (%)	Time (s)	Obj	Gap (%)	Time (s)	Obj	Gap (%)	Time (s)	Obj	Time (s)
rl5915	5915	607557	6.20%	1.9	**573246**	**0.20%**	2.3	1229421	114.90%	147.6	644094	12.59%	2.6	572085	594
rl5934	5934	608666	8.75%	2.1	**565169**	**0.98%**	2.2	1164891	108.12%	149.2	585930	4.68%	3.0	559712	594
pla7397	7397	25637656	9.65%	5.1	**23556504**	**0.75%**	4.3	58497847	150.18%	290.7	24855608	6.30%	4.0	23382264	744
rl11849	11849	1004531	8.13%	13.1	**944262**	**1.64%**	13.0	2071487	122.98%	1184.9	979228	5.41%	14.5	929001	1134
usa13509	13509	22210920	10.32%	19.7	**20340434**	**1.03%**	16.7	–	–	–	21785452	8.20%	18.5	20133724	1356
brd14051	14051	508689	7.28%	20.1	**473204**	**-0.20%**	17.8	–	–	–	473336	-0.17%	16.9	474149	1410
d15112	15112	1706147	7.40%	23.9	**1582514**	**-0.38%**	25.2	–	–	–	1593474	0.31%	23.9	1588550	1512
d18512	18512	691892	5.97%	41.1	**648584**	**-0.66%**	32.6	–	–	–	653900	0.15%	34.5	652911	1860
pla33810	33810	71908968	7.19%	72.3	**67577760**	**0.74%**	74.5	–	–	–	72764816	8.47%	76.0	67084217	3384
pla85900	85900	154645568	7.39%	631.5	**143791392**	**-0.15%**	928.8	–	–	–	144612192	0.42%	996.1	144004484	23400

Table 2. Results on Euclidean TSP with test size $n = 500$. Error is reported as a percentage gap to LKH-3 (lower is better).

Method	Uses LKH	Gap (%)	Time (s)
LKH-3	Yes	0.0000	14.658
Ours + LKH	Yes	**0.5225**	**0.954**
NeuroLKH	Yes	2.6961	1.207
Ours + Greedy	No	6.9720	2.910
POMO	No	30.7130	2.476

Table 3. Results on Euclidean TSP with test size $n = 2000$. Error is reported as a percentage gap to LKH-3 (lower is better).

Method	Uses LKH	Gap (%)	Time (s)
LKH-3	Yes	0.0000	327.022
Ours + LKH	Yes	**0.5840**	**4.406**
NeuroLKH	Yes	0.7205	5.452
Ours + Greedy	No	7.7242	10.316
POMO	No	62.2305	41.699

any preprocessing and I/O) to the production of the final tour. Each method is executed with its default configuration. We train our model only on synthetic Euclidean TSP instances generated by the standard **attention-learn-to-route** protocol (Kool et al., 2018): $n = 50$ points uniformly sampled in the unit square (50,000 instances). We then evaluate *without retraining* on (i) synthetic instances with $n \in \{500, 2000\}$ (a $10\times$–$40\times$ size increase), and (ii) real-world **TSPLIB** (Reinelt, 1991) instances up to $n = 85{,}900$ (a $1718\times$ size increase). For synthetic tests, we use LKH-3 to obtain the reference tour length $\mathrm{len}_{\mathrm{ref}}$; for **TSPLIB**, we use the LKH objectives reported by **DualOpt** (Zhou et al., 2025) as references. We focus on the 10 largest TSPLIB instances in Table 1 (from 5,915 to 85,900 nodes) to stress-test size generalization. We report the optimality gap $\mathrm{Gap}(\%) = 100 \times \frac{\mathrm{len}-\mathrm{len}_{\mathrm{ref}}}{\mathrm{len}_{\mathrm{ref}}}$.

Our model predicts sparse edge scores on a geometric spanner, which we use in two solver variants. **Ours+Greedy**: As an ablation that removes Lin–Kernighan post-processing, we apply a lightweight 2-opt refinement after the top-down decoding (Appendix A.3). **Ours+LKH**: We form per-node candidate lists (top-$k = 20$), map scores to LKH candidate costs, and run warm-started LKH-3. We use a lightweight hierarchical attention backbone with width $d = 128$. Training uses AdamW (lr 10^{-4}, weight decay 10^{-4}), batch size 8, for 20 epochs; teacher tours are produced by LKH-3 by default. We compare against: (i) **LKH-3** (Helsgaun, 2017) as a strong heuristic reference, (ii) **NeuroLKH** (Xin et al., 2021) as a learning-to-candidate baseline coupled with LKH, and (iii) **POMO** (Kwon et al., 2020) as a constructive neural baseline with its standard evaluation setup.

Results on synthetic datasets. Trained only on $n = 50$, our method generalizes reliably to much larger instances: $n = 500$ ($10\times$) and $n = 2000$ ($40\times$) in Tables 2–3. **Ours+LKH** achieves a 0.52% gap in 0.95s at $n = 500$ and a 0.58% gap in 4.41s at $n = 2000$, outperforming **NeuroLKH** in both quality and runtime, while reducing the wall-clock time of **LKH-3** by about $15\times$ and $74\times$, respectively. The ablation **Ours+Greedy** remains fast but less accurate, whereas constructive baselines such as **POMO** degrade sharply as n grows (e.g., 30.71% at $n = 500$ and 62.23% at $n = 2000$), highlighting the benefit of our size-independent guidance under extrapolation.

Results on TSPLIB. Table 1 shows that the same model trained on $n = 50$ transfers to large-scale TSPLIB instances spanning 5,915 to 85,900 nodes ($1718\times$ larger). **Ours+LKH** consistently matches the LKH-3 reference with gaps in $[-0.66\%, 1.64\%]$ (often even negative), while reducing wall-clock time from 594–23,400s to 2.2–928.8s (a $25\times$–$270\times$ speedup), underscoring the stronger size generalization enabled by our size-independent interface.

5. Conclusion

We proposed a size-independent neural dynamic programming approach for Euclidean TSP that learns only the DP merge operation through a fixed-size boundary interface, while preserving the PTAS-style $(1+\varepsilon)$-approximation guarantee. We also provided a local-to-global analysis showing that small merge errors lead to a controlled overall approximation. The experiments demonstrate strong cross-size generalization and practical benefits when combined with classical heuristics.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. *The traveling salesman problem: a computational study*. Princeton university press, 2011.

Arora, S. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *J. ACM*, 45(5):753–782, September 1998. ISSN 0004-5411. doi: 10.1145/290179.290180.

Bartal, Y. and Gottlieb, L.-A. A linear time approximation scheme for euclidean tsp. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, pp. 698–706. IEEE, 2013.

Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

Cook, W. World traveling salesman problem. URL https://www.math.uwaterloo.ca/tsp/world/.

Fu, Z.-H., Qiu, K.-B., and Zha, H. Generalize a small pre-trained model to arbitrarily large tsp instances. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 7474–7482, 2021.

Har-Peled, S. *Geometric approximation algorithms*. Number 173. American Mathematical Soc., 2011.

Helsgaun, K. An effective implementation of the lin–kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000. ISSN 0377-2217. doi: https://doi.org/10.1016/S0377-2217(99)00284-2.

Helsgaun, K. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems: Technical report, 2017.

Hudson, B., Li, Q., Malencia, M., and Prorok, A. Graph neural network guided local search for the traveling salesperson problem. 2021.

Johnson, D. S. and McGeoch, L. A. The traveling salesman problem: A case study in local optimization. volume 1, pp. 215–310. Chichester, UK, 1997.

Kahng, A. B., Nerem, R. R., Wang, Y., and Yang, C.-Y. Nn-steiner: A mixed neural-algorithmic approach for the rectilinear steiner minimum tree problem. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 13022–13030, 2024.

Kool, W., Van Hoof, H., and Welling, M. Attention, learn to solve routing problems! 2018.

Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., and Min, S. Pomo: Policy optimization with multiple optima for reinforcement learning. volume 33, pp. 21188–21198, 2020.

Li, Y., Guo, J., Wang, R., and Yan, J. T2t: From distribution learning in training to gradient search in testing for combinatorial optimization. volume 36, pp. 50020–50040, 2023.

Li, Y., Guo, J., Wang, R., Zha, H., and Yan, J. Fast t2t: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. volume 37, pp. 30179–30206, 2024.

Lin, S. and Kernighan, B. W. An effective heuristic algorithm for the traveling-salesman problem. *Oper. Res.*, 21: 498–516, 1973.

Lykouris, T. and Vassilvitskii, S. Competitive caching with machine learned advice. volume 68, pp. 1–25. ACM New York, NY, 2021.

McCarty, E., Zhao, Q., Sidiropoulos, A., and Wang, Y. NN-Baker: A neural-network infused algorithmic framework for optimization problems on geometric intersection graphs. In *Advances in Neural Information Processing Systems*, pp. 23023–23035, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/c236337b043acf93c7df397fdb9082b3-Abstract.html.

Min, Y., Bai, Y., and Gomes, C. P. Unsupervised learning for solving the travelling salesman problem. volume 36, pp. 47264–47278, 2023.

Mitchell, J. S. B. Guillotine subdivisions approximate polygonal subdivisions: a simple new method for the geometric k-mst problem. pp. 402–408, 1996.

Rao, S. B. and Smith, W. D. Approximating geometrical graphs via "spanners" and "banyans". In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, STOC '98, pp. 540–550, New York, NY, USA, 1998. Association for Computing Machinery. ISBN 0897919629. doi: 10.1145/276698.276868.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Reinelt, G. Tsplib - a traveling salesman problem library. *INFORMS J. Comput.*, 3:376–384, 1991.

Sun, Z. and Yang, Y. Difusco: Graph-based diffusion solvers for combinatorial optimization. volume 36, pp. 3706–3731, 2023.

Veličković, P. and Blundell, C. Neural algorithmic reasoning. *Patterns*, 2(7), 2021.

Vinyals, O., Fortunato, M., and Jaitly, N. Pointer networks. volume 28, 2015.

Xin, L., Song, W., Cao, Z., and Zhang, J. Neurolkh: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. volume 34, pp. 7472–7483, 2021.

Zhou, S., Ding, Y., Zhang, C., Cao, Z., and Jin, Y. Dualopt: a dual divide-and-optimize algorithm for the large-scale traveling salesman problem. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'25/IAAI'25/EAAI'25. AAAI Press, 2025. ISBN 978-1-57735-897-8. doi: 10.1609/aaai.v39i25.34926.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

A. Additional Design Details for the DP and Practical Variants

A.1. A deterministic PARSE design for the certified DP variant

In the certified DP view, each child box B is summarized by a discrete interface state $\sigma_B \in \Omega(B)$ that specifies how the global tour interacts with the boundary of B. Our learned merge primitive produces a continuous proxy $\tilde{\sigma}_B \in \mathbb{R}^m$, which must be decoded to a legal discrete state for exact feasibility checking and exact DP evaluation. This appendix records a concrete deterministic PARSE design (not implemented in our engineering variant).

Interface parameterization. We assume a fixed catalog of boundary decisions with bounded size under r-lightness: (i) *portal activations* on each side of ∂B, and (ii) a *non-crossing connectivity pattern* among the activated portals in clockwise boundary order. This matches the standard Arora/Rao-style DP semantics, where the tour crosses box boundaries only at portals and the induced connectivity on ∂B is planar.

Step 1: portal selection under constraints. Let $\pi_j(\tilde{\sigma}_B) \in (0, 1)$ denote the activation score (after sigmoid) of portal j. A deterministic portal set can be obtained by thresholding or top-k selection per side, followed by hard constraint projection:

- **Budget:** enforce the r-light limit (or its per-side equivalent) by keeping the highest-scoring portals.
- **Parity/degree sanity:** enforce a feasible number of boundary incidences consistent with a Hamiltonian tour (e.g., evenness of boundary endpoints for the induced pairing model, and excluding obviously impossible counts).

If the projected set violates basic feasibility (e.g., no feasible connectivity exists), we fall back to a nearby set by minimally adjusting portal activations (e.g., adding/removing the lowest-confidence portals).

Step 2: planar connectivity decoding via constant-size DP. Given the selected portals in clockwise order, the remaining task is to decode a planar connectivity pattern that is legal under the DP definition of $\Omega(B)$. Because the number of active portals is $O(r)$ (a small constant for fixed ε), we can solve this exactly using a constant-size dynamic program (or equivalently, a minimum-cost non-crossing matching DP). We define per-connection costs from logits inside $\tilde{\sigma}_B$ and select the minimum-cost legal pattern:

$$\sigma_B \;=\; \arg \min_{\sigma \in \Omega(B) \text{ consistent with selected portals}} \mathrm{Cost}(\sigma; \tilde{\sigma}_B).$$

This yields a deterministic PARSE that (i) produces a legal state when one exists and (ii) exposes an explicit feasibility check.

Top-K decoding (optional). To reduce fallback frequency in edge cases, PARSE may emit the top-K feasible states under $\mathrm{Cost}(\cdot; \tilde{\sigma}_B)$. Downstream DP then selects the best state among these candidates while remaining fully discrete and verifiable.

Remark. Our engineering instantiation bypasses exact DP tables and does not require PARSE; we record this design to clarify how the learned continuous representation can be snapped to certified DP semantics.

A.2. Leaf initialization: how leaf latents h_B are obtained

The two-pass practical implementation assumes a cached latent vector h_B for every box B. For internal nodes, h_B is produced by aggregating children. For *leaf* boxes (no children), we instantiate h_B directly from local inputs.

Tokenization. Let c_B be the center of box B and $\ell(B)$ its side length. For each point $p \in P \cap B$, we form a point token embedding

$$x_p \;=\; \phi_p\left(\left[\frac{p - c_B}{\ell(B)};\; 1\right]\right),$$

where ϕ_p is a small MLP and the constant 1 serves as a type indicator. We also form a small set of geometry tokens from $Geom(B)$ (e.g., normalized box coordinates, depth, or side length encodings), embedded by another MLP ϕ_g.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Encoder and pooling. We apply the same lightweight attention encoder used elsewhere (or a shared block with separate input projections) to the concatenation of geometry tokens and point tokens. We then pool to obtain the leaf latent:

$$h_B \;=\; \mathrm{Pool}\big(\mathrm{Enc}(\{x_p\}_{p\in P\cap B}, \{\phi_g(Geom(B))\})\big),$$

where Pool is either a **[CLS]** token readout or mean pooling.

Empty leaves. If $P \cap B = \varnothing$, we set h_B using only geometry tokens plus a learned **[EMPTY]** token, so that latents remain well-defined for all boxes.

A.3. Lightweight greedy post-processing as an ablation of LKH post-processing

Our practical pipeline can hand off a warm-start tour and learned candidate guidance to LKH-3. To isolate the effect of learned guidance from the heavier Lin–Kernighan search, we also consider a purely lightweight greedy post-processing variant used in our ablations.

Candidate-restricted 2-opt. Given an initial tour T (from the lightweight constructor), we repeatedly apply 2-opt moves that reduce the tour length. To keep the procedure fast and aligned with the learned sparsity bias, we restrict candidate edges introduced by a 2-opt move to lie in a sparse set (e.g., the patched spanner $E_{S'}$ or the NN candidate set induced by the learned scores). Concretely, we only consider swaps that introduce edges (a, c) and (b, d) such that both appear in the candidate set.

Procedure. We run a small, fixed number of passes, applying either the first improving move or the best move per pass. This yields a refinement that is substantially cheaper than Lin–Kernighan while still capturing easy local improvements. In experiments, the resulting solver is reported as **Greedy** and should be interpreted as a post-processing ablation that replaces LKH's downstream search with a lightweight heuristic.

A.4. Practical instantiation details

We provide the full engineering instantiation of the two-pass schedule used for fast guidance, including the exact bottom-up construction, the top-down propagation rule, and how continuous boundary-state predictions are converted into edge-level signals for downstream heuristics.

Step 1: Bottom-up encoding. We first run a bottom-up encoding pass to compute and cache a latent vector h_B for every box. Compared to the theoretical variant, the bottom-up encoder here uses the *children's latents* to form h_B: h_B is computed by aggregating $\{h_{B_i}\}$ rather than encoding the discrete child states $\{C_{B_i}\}$. See Fig. 3a for the exact bottom-up construction. For leaf boxes, h_B is produced by a lightweight leaf encoder over local geometry and contained points; we defer the exact construction to Appendix A.2.

Step 2: Top-down decoding. We then perform a top-down pass, analogous to DP traceback: each box B is visited exactly once under a single continuous conditioning state $\tilde{\sigma}_B$ (the boundary state predicted by B's parent), and a single decoder call produces $\tilde{\tau}(\tilde{\sigma}_B) = (\tilde{\sigma}_{B_1}, \tilde{\sigma}_{B_2}, \tilde{\sigma}_{B_3}, \tilde{\sigma}_{B_4})$, which we propagate to the four children. Figure 3 summarizes this two-pass schedule.

Fast guidance mode (no DP tables). In the practical pipeline we do *not* materialize full DP cost tables, hence we bypass the PARSE/exact-evaluation/fallback chain. Instead, the top-down pass produces a collection of continuous boundary states, including real-valued *crossing logits* on the fixed local boundary crossing representation (determined only by $r(\varepsilon)$), which we reduce to edge-level guidance for a downstream heuristic solver.

From crossing logits to spanner-edge scores. Let $G_{S'} = (V, E_{S'})$ be the r-light patched spanner from the PTAS backbone (so $|E_{S'}| = O(n)$). We aggregate the predicted crossing logits into a single score s_e for each spanner edge $e \in E_{S'}$ (using an edge-id based reduction such as mean/max pooling over the coordinates mapped to e), and set s_e to a very small value when e is not covered by any crossing coordinate. This yields a sparse score vector $s \in \mathbb{R}^{|E_{S'}|}$ over spanner edges.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Warm start via a lightweight constructor. We build a feasible tour using a lightweight constructive routine on the spanner. Concretely, we sort spanner edges by decreasing s_e and greedily add edges subject to degree-≤ 2 constraints and no premature subtours. If the greedy result forms multiple path components, we optionally patch them by adding a few off-spanner edges (connecting nearest component endpoints) to obtain a Hamiltonian cycle. When successful, this tour serves as an initial solution for the strong heuristic below. We also consider a lightweight greedy post-processing (candidate-restricted 2-opt) that replaces LKH's downstream search as an ablation; details are in Appendix A.3.

Guiding LKH with NN candidates. We run LKH-3 on the original Euclidean instance (EUC_2D), i.e., the objective remains the Euclidean tour length. Our scores only influence the search through (i) a candidate set and (ii) an optional warm start. From $E_{S'}$ we construct an LKH candidate list by selecting, for each node u, the top-K incident neighbors v with the highest scores $s_{(u,v)}$. We convert scores into LKH **alpha**-values via

$$\alpha(u,v) = \lambda\Big(\max_{e \in E_{S'}} s_e - s_{(u,v)}\Big),$$

so higher-score edges receive smaller (better) **alpha**. We pass this candidate file to LKH together with the warm-start tour.

Training (overview). We train the shared encoder–decoder to predict continuous boundary states $\tilde{\sigma}$ used throughout the practical instantiation (Section 3.3). Supervision is derived from high-quality teacher tours on small Euclidean instances ($n = 50$): for each visited box B under conditioning state $\tilde{\sigma}_B$, we construct binary targets indicating which boundary crossing decisions are used by the teacher tour. We optimize a masked binary cross-entropy objective on the corresponding real-valued state coordinates (after an elementwise sigmoid), and train under free-running conditioning to match the inference schedule. Full details are deferred to Appendix G.

B. Other Related Works

Geometric PTAS backbones for Euclidean TSP. The Euclidean TSP admits polynomial-time approximation schemes (PTAS) based on hierarchical spatial decompositions and dynamic programming over boundary states. Classic schemes include Arora's randomized PTAS built on shifted quadtrees and portals (Arora, 1998), and Mitchell's m-guillotine framework that yields PTASs for geometric TSP and related problems (Mitchell, 1996). Rao and Smith proposed a closely related spanner/patching perspective for low-dimensional Euclidean spaces (Rao & Smith, 1998), which has been influential for organizing local computations with certifiable global guarantees. Subsequent work improved the running time dependence on the number of points, including linear-time approximation schemes for Euclidean TSP in fixed dimension (Bartal & Gottlieb, 2013). Despite their strong guarantees, PTAS frameworks are rarely used as general-purpose solvers in practice, in large part due to substantial hidden constants and implementation complexity of the DP routines (see also the broader computational study literature (Applegate et al., 2011)).

Neural-accelerated dynamic programs and learning-augmented algorithms. A complementary line of research studies how learning can be inserted into classical algorithmic pipelines without discarding the surrounding algorithmic structure. At a high level, the "learning-augmented" viewpoint treats predictions as advice that can improve performance while retaining worst-case robustness when the advice is inaccurate (Lykouris & Vassilvitskii, 2021). More broadly, neural algorithmic reasoning asks how neural networks can emulate or adapt algorithmic computations to improve generalization and efficiency (Veličković & Blundell, 2021). Within combinatorial optimization, surveys have highlighted a recurring pattern: instead of learning entire solvers, learning is often most effective when it targets the expensive or ill-specified decision subroutines inside strong optimization backbones (Bengio et al., 2021). Our work follows this principle in a PTAS setting: we keep the certified geometric backbone, and learn only the DP *merge* primitive (the practical bottleneck), with explicit constraints ensuring composability across levels and compatibility with end-to-end approximation guarantees.

Modern neural and hybrid TSP solvers. Learning-based TSP solvers are typically evaluated as fast heuristics, and can be grouped by how they produce tours and how tightly they couple to classical optimization.

Auto-regressive (AR) construction. Early neural solvers generate tours sequentially. Pointer Networks introduced attention-based decoding for variable-size combinatorial outputs and demonstrated feasibility on planar TSP variants (Vinyals et al., 2015). A major step for routing problems is the attention model of Kool et al. (2018), which uses a Transformer-style architecture and policy-gradient training to construct tours node-by-node. Kwon et al. (2020) proposed POMO, exploiting

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

symmetries via multiple rollout starts to stabilize RL training and improve solution quality. AR models often achieve strong tour quality, but sequential decoding can limit throughput on large instances and complicate tight latency budgets.

Non-auto-regressive (NAR) generation and refinement. To mitigate sequential decoding, recent work explores parallel generation and iterative refinement. Diffusion-based approaches such as DIFUSCO treat tours as structured objects and denoise towards high-quality solutions (Sun & Yang, 2023). The Training-to-Testing (T2T) framework leverages a generative model during training and performs gradient-based search at test time to refine solutions (Li et al., 2023), with follow-up work improving efficiency via optimization-consistency objectives to reduce refinement steps (Li et al., 2024). NAR methods can better exploit parallel hardware, but must carefully enforce feasibility and often rely on refinement schedules whose effectiveness can vary with instance size and distribution.

Hybrid pipelines. Classical local search remains extremely strong for Euclidean TSP, with Lin–Kernighan style variable-depth k-opt search and modern implementations achieving excellent performance in practice (Lin & Kernighan, 1973; Helsgaun, 2000). Hybrid methods combine such pipelines with learned components that provide guidance (e.g., candidate edges, edge penalties, or initialization), aiming to inherit robustness from optimization while gaining speed or quality from learning. NeuroLKH integrates learned edge scores into the Lin–Kernighan–Helsgaun heuristic (Xin et al., 2021). GNN-GLS predicts edge regret to guide guided local search (Hudson et al., 2021). UTSP learns edge heatmaps with an unsupervised objective and couples them with search to produce tours (Min et al., 2023). These hybrids are powerful heuristics, but they typically do not provide PTAS-style end-to-end approximation guarantees. Our approach is orthogonal: it learns a constrained DP subroutine inside a geometric PTAS backbone, thereby targeting practical acceleration while preserving a certifiable algorithmic wrapper.

C. PTAS backbone and structural restriction

This appendix summarizes the classic PTAS backbone for Euclidean TSP and clarifies the specific restriction that yields a *constant-size interface* for our DP. Our presentation follows the high-level blueprint of Arora (Arora, 1998) and the spanner-guided refinement of Rao–Smith (Rao & Smith, 1998).

C.1. Well-rounded instances and randomized quadtree dissection

As in Arora (Arora, 1998), we assume a *well-rounded* instance after a standard perturbation/rescaling step: (i) coordinates are integral on a grid, (ii) the minimum nonzero interpoint distance is a fixed constant (e.g., 8), and (iii) the bounding box is a square of side length L (polynomially bounded in n). We then pick a random shift (a, b) and construct the shifted dissection (quadtree) by recursively subdividing squares until each leaf contains at most one point.

The random shift is crucial for the averaging argument in the structure theorem: the probability that an edge (or a tour segment) is cut at a given scale decreases geometrically with the scale, enabling the expected cost increase due to local modifications to be charged to grid lines/hyperplanes.

C.2. Arora's structure theorem: portal-respecting (m, r)-light tours

Arora proves a *Structure Theorem* stating that for any accuracy parameter $\varepsilon > 0$ (equivalently $c = \Theta(1/\varepsilon)$), with probability at least $1/2$ over the random shift, there exists a near-optimal tour that is (m, r)-*light* with respect to the shifted dissection, where $m = O(c \log L)$ portals per side are placed at prespecified locations and $r = O(c)$ bounds the number of boundary crossings per side (Arora, 1998). The theorem has two conceptual components:

1. **Crossing bound:** there is a $(1 + O(\varepsilon))$-approximate tour that crosses each square boundary only $O(1/\varepsilon)$ times (after patching).

2. **Portal restriction:** moreover, these crossings can be forced to occur only at a small set of portals on each boundary, by allowing short detours whose expected total cost is $O(\varepsilon) \cdot \mathrm{OPT}$.

This restriction yields a DP state for each square consisting of (i) a multiset of boundary portals used by the tour, and (ii) a noncrossing pairing describing how the partial solution inside the square connects these boundary endpoints. Arora calls the resulting subproblem the (m, r)-*multipath* problem and builds the DP bottom-up across the quadtree (Arora, 1998).

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

C.3. Rao–Smith refinement: spanner-guided lightness without uniform portals

Rao–Smith (Rao & Smith, 1998) observe that the *portal grid* is not the only way to obtain a constant-size boundary interface. Their key idea is to:

1. construct a sparse geometric graph S (a $(1+\varepsilon)$-spanner) on the point set, with near-linear size and bounded degree; and then
2. *patch/modify* this graph with respect to the same shifted dissection to obtain a graph S' whose number of crossings with each quadtree line/box boundary is bounded by a function of ε only (an "r-light" graph).

They show (informally: a structural restriction theorem) that with constant probability over the shift, one can obtain an r-light graph that is *guaranteed to contain* a tour whose length is within $(1+O(\varepsilon))$ of optimal (Rao & Smith, 1998). The subsequent optimization problem becomes: *find the best tour within* S', subject to the r-light restriction on crossings. Because S' is sparse and r depends only on ε, the same Arora-style DP can be run with a constant-size interface, but now the interface points are the *actual spanner–boundary intersections* rather than evenly spaced portals.

C.4. A formal structure lemma (spanner-guided r-light restriction)

We restate the classic structural guarantee (Arora, 1998) in the spanner-guided form of Rao–Smith (Rao & Smith, 1998), specialized to our interface definition $X_B = \partial B \cap E(S')$ and legal family $\mathcal{S}_{legal}$.

Setup. Fix $\varepsilon \in (0,1)$. Let OPT be the length of an optimal Euclidean TSP tour. Consider the following randomized construction: (1) apply the standard well-rounded preprocessing (scaling/perturbation) as in (Arora, 1998); (2) sample a random shift for the quadtree dissection; (3) build a $(1+\varepsilon_s)$-spanner S on the points; (4) apply the Rao–Smith patching procedure w.r.t. the shifted dissection to obtain a patched spanner S' that is r-light for $r = r(\varepsilon)$ (at most r crossings per side of every dissection square/box).

The parameters $\varepsilon_s, \varepsilon_p > 0$ are chosen such that $\varepsilon_s + \varepsilon_p \le \varepsilon/3$ (one convenient choice is $\varepsilon_s = \varepsilon_p = \varepsilon/6$), so that spanner stretch and patching overhead fit in the final approximation budget.

Lemma C.1 (Spanner-guided r-light structure)**.** *For any $\varepsilon \in (0,1)$, there exist constants $r = r(\varepsilon)$ and $c > 0$ such that the following holds. With probability at least c over the random shift of the dissection, the patched spanner S' contains a tour $T^\star$ satisfying:*

1. ***Near-optimality:*** $\mathrm{len}(T^\star) \le (1+O(\varepsilon)) \cdot \mathrm{OPT}$.
2. ***r-light boundary interaction:*** *for every quadtree box B, the tour $T^\star$ crosses each side of ∂B at most r times; equivalently, $|X_B| = |\partial B \cap E(S')| \le 4r$ and all boundary crossings of $T^\star$ occur at sites in X_B.*
3. ***Legality:*** *$T^\star \in \mathcal{S}_{legal}$, i.e., for every box B, the restriction of $T^\star$ to B is a disjoint union of paths embedded in S' whose endpoints are boundary sites in X_B, inducing a planar noncrossing pairing on those endpoints.*

Proof sketch (following (Arora, 1998)). We outline the three standard transformations and their cost accounting.

(i) Spanner containment with small stretch. Rao–Smith show that a sparse geometric spanner can be constructed so that there exists a tour in the spanner whose length is at most $(1+O(\varepsilon_s)) \cdot \mathrm{OPT}$ (Rao & Smith, 1998). Let T_S denote such a tour in S.

(ii) Patching to enforce r-lightness. Given the shifted dissection, Rao–Smith patch the spanner (by local modifications near dissection boundaries) to obtain S' such that every dissection line/box boundary is intersected only $r = r(\varepsilon)$ times per side, while preserving spanner-like distance guarantees and increasing the length of the best tour in the graph by at most an additive $O(\varepsilon_p) \cdot \mathrm{OPT}$ in expectation over the random shift (Rao & Smith, 1998). This yields a tour $T_{S'} \subseteq S'$ with $\mathrm{len}(T_{S'}) \le (1+O(\varepsilon_s+\varepsilon_p)) \cdot \mathrm{OPT}$.

(iii) From r-lightness to a constant-size interface and legality. Since $T_{S'}$ is a subgraph of S', every time it crosses a box boundary ∂B it must do so along an edge of S' that intersects ∂B, hence at a site in $X_B = \partial B \cap E(S')$ by definition. The r-light property bounds $|X_B|$ by a function of ε only, giving a constant-size boundary interface. Finally, the standard planar

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

multipath argument (as in Arora's (m, r)-multipath restriction (Arora, 1998)) implies that within each box, the tour induces a disjoint union of interior path segments whose boundary endpoints admit a planar noncrossing pairing. This is exactly the legality condition of $\mathcal{S}_{legal}$. Combining (i)–(iii) and choosing $\varepsilon_s + \varepsilon_p \leq \varepsilon/3$ gives the stated approximation. □

Corollary C.2 (DP optimality within $\mathcal{S}_{legal}$ yields a PTAS)**.** *Assume the DP computes the optimal tour within $\mathcal{S}_{legal}$ exactly (i.e., it minimizes over all boundary states and feasible child tuples). Then, conditioned on the event in Lemma C.1, the DP output has length at most $(1 + O(\varepsilon)) \cdot \mathrm{OPT}$. In particular, the interface/state space size depends only on $r(\varepsilon)$ (hence only on ε), not on n.*

Amplifying success probability. As in (Arora, 1998), the constant success probability over the random shift can be amplified by trying $O(\log(1/\delta))$ independent shifts and taking the best result, yielding success probability at least $1 - \delta$.

C.5. Our legal family $\mathcal{S}_{legal}$ and the constant-size interface

We adopt the Rao–Smith viewpoint in defining our restricted family $\mathcal{S}_{legal}$. Fix the patched spanner S' produced from the input instance and the random quadtree. For each box B in the quadtree, let

$$X_B := \partial B \cap E(S')$$

be the set of spanner edges intersecting the boundary. The r-light property implies $|X_B| \leq 4r$ (at most r crossings per side).

A *legal* partial solution inside a box B is a disjoint union of paths embedded in S' whose endpoints lie in X_B, which together visit all points inside B, and which induce a planar noncrossing pairing on the boundary endpoints. This is the spanner analogue of Arora's multipath restriction; it ensures that each DP state has constant-size encoding independent of n.

Consequently, the DP can be formulated over the *boundary state space* $\Omega(B)$ defined in Appendix D, with interface dimension $4r$. The correctness argument follows the classic template:

1. By the structural restriction (Arora-style patching / Rao–Smith spanner-guided patching), with constant probability there exists a $(1 + O(\varepsilon))$-approximate tour in $\mathcal{S}_{legal}$.
2. The DP computes the optimum solution within $\mathcal{S}_{legal}$ exactly.
3. Therefore, the DP output is a $(1 + O(\varepsilon))$-approximation to OPT.

C.6. Algorithmic skeleton

For completeness, the end-to-end skeleton is:

1. **Preprocess:** rescale/perturb to obtain a well-rounded instance.
2. **Dissection:** sample (or search over) a quadtree shift and build the shifted quadtree.
3. **Sparse guide:** build a $(1 + \varepsilon)$-spanner S on the points.
4. **Patch to lightness:** modify/patch S with respect to the dissection to obtain S' that is r-light for $r = r(\varepsilon)$.
5. **DP:** compute the optimal legal solution in S' via bottom-up merges over quadtree states $\Omega(B)$.

The only departure from Arora's original portal-respecting DP is the definition of the interface: we use $X_B = \partial B \cap E(S')$ as the canonical crossing sites (Appendix D), instead of prespecified, equally spaced portals.

D. Boundary states: definition and encoding

This appendix makes the DP state space $\Omega(B)$ explicit and provides a concrete, reproducible encoding. Throughout, we work under the r-light restriction induced by the patched spanner S': for every box $B \in \mathcal{T}$, the boundary ∂B is intersected by S' at most r times per side and at most $4r$ times in total.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

D.1. Canonical crossing sites and a fixed interface

For each box B, define the set of *crossing sites*

$$X_B := \partial B \cap E(S'), \qquad m_B := |X_B| \le 4r,$$

where each element is a geometric point $x \in \partial B$ at which a straight spanner edge intersects the boundary. These points (and their normalized coordinates) are precisely the "boundary geometry" consumed by the merge primitive via $Geom(B)$.

Clockwise order. Let $\prec_B$ be the clockwise cyclic order of points on ∂B, breaking ties arbitrarily (there are none w.p.1 under generic perturbations). Write

$$x_1 \prec_B x_2 \prec_B \cdots \prec_B x_{m_B}$$

for the ordered list of sites.

Slotting to a fixed dimension. To obtain a *uniform* interface across all boxes and levels, we assign each potential site to one of $4r$ slots. Concretely, each side of B has at most r sites; we sort the sites on a side by their coordinate along that side and place them into slots (s, t) where $s \in \{\mathsf{N}, \mathsf{E}, \mathsf{S}, \mathsf{W}\}$ and $t \in \{1, \dots, r\}$. If a side has fewer than r sites, the remaining slots are marked *empty*. We thus obtain a partial bijection

$$\mathrm{slot}_B : X_B \to [4r],$$

and a mask $M_B \in \{0,1\}^{4r}$ that indicates which slots are present. All state encodings below are defined over the fixed index set $[4r]$, with empty slots masked out.

D.2. Discrete boundary states for TSP

A boundary state $\sigma \in \Omega(B)$ describes how a partial solution inside B interacts with ∂B. Under the legal-family restrictions, edges can cross ∂B only at crossing sites, so the boundary interaction is captured by (i) which sites are used, and (ii) how they are connected within B.

Active crossings. Let $A(\sigma) \subseteq X_B$ be the set of *active* boundary crossings used by the partial solution inside B. Equivalently, in slotted form we store a binary vector

$$a(\sigma) \in \{0,1\}^{4r} \quad \text{with} \quad a_p(\sigma) = 1 \;\Leftrightarrow\; \exists x \in A(\sigma) : \mathrm{slot}_B(x) = p,$$

and we enforce $a_p(\sigma) = 0$ whenever $M_{B,p} = 0$ (empty slots).

For TSP, every time the tour crosses ∂B it must also leave B, hence $|A(\sigma)|$ is even. We therefore require $\sum_{p=1}^{4r} a_p(\sigma)$ to be even.

Noncrossing pairing (connectivity). Given the clockwise order $\prec_B$, the induced connectivity of the partial solution inside B can be represented by a *planar noncrossing pairing* on the active sites: each active site is an endpoint of exactly one path segment inside B, and active sites are paired by which two endpoints belong to the same path. Formally, let $A := A(\sigma)$ and write the active sites in clockwise order as

$$y_1 \prec_B y_2 \prec_B \cdots \prec_B y_{2k}, \qquad 2k = |A|.$$

A pairing is an involution $\pi : [2k] \to [2k]$ with no fixed points, i.e., $\pi(i) \neq i$ and $\pi(\pi(i)) = i$. It is *noncrossing* if there do not exist indices $i < j < \pi(i) < \pi(j)$ (after interpreting the order on a circle with a fixed cut; equivalently, chords between matched points do not intersect).

We encode the pairing by a partner map in slot indices: for each active slot p, store $\mathrm{mate}_\sigma(p) = q$ where q is the slot of the paired site. Empty and inactive slots store $\mathrm{mate}_\sigma(p) = \perp$.

Summary. A discrete state is thus

$$\sigma \equiv \big(a(\sigma), \mathrm{mate}_\sigma(\cdot)\big),$$

subject to: (i) slot mask constraints ($a_p = 0$ for empty slots), (ii) even parity of active crossings, and (iii) noncrossing consistency of the induced pairing under $\prec_B$.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

D.3. State-space size bound (one-line proof)

For completeness we restate why $|\Omega(B)|$ depends only on r. There are at most $2^{m_B} \le 2^{4r}$ choices of the active subset. Conditioned on $|A| = 2k$, the number of noncrossing pairings is the Catalan number $\mathrm{Cat}_k \le 4^k$. Hence

$$|\Omega(B)| \le \sum_{k=0}^{2r} \binom{4r}{2k} \mathrm{Cat}_k \le 2^{4r} \cdot 4^{2r} = 2^{8r},$$

which is $r^{O(r)}$ and therefore constant for fixed ε (since $r = r(\varepsilon)$).

D.4. Remarks: component labels and other legal families

The above definition matches the common TSP dissection DP view where the interior solution is a disjoint union of paths whose endpoints lie on ∂B. If the chosen legal family $\mathcal{S}_{legal}$ requires additional global-connectivity bookkeeping (e.g., to rule out premature subtours), the state can be extended in a standard way by attaching a constant-size component/label structure to the pairing (still bounded by $r^{O(r)}$). Our implementation keeps the *same* fixed interface $[4r]$ and adds only $O(r)$ extra discrete symbols per state.

E. Feasible tuples and consistency rules

This appendix defines the feasibility set $\mathcal{F}_B(\sigma)$ used by the DP recurrence

$$c[B, \sigma] \;=\; \min_{\tau \in \mathcal{F}_B(\sigma)} \sum_{i=1}^{4} c[B_i, \sigma_{B_i}],$$

where B is an internal quadtree box with four children $B_1, \ldots, B_4$ and $\tau = (\sigma_{B_1}, \ldots, \sigma_{B_4})$ is a 4-tuple of child boundary states. The definition is the spanner analogue of Arora's (m, r)-multipath composition rules: children must agree on shared boundaries and together induce the parent boundary connectivity in a planar/noncrossing manner (Arora, 1998).

E.1. Geometric correspondence maps between interfaces

Each box uses the fixed $4r$-slot interface described in Appendix D.1. To compose children into a parent, we precompute three kinds of deterministic maps:

(1) Parent-to-child map on the outer boundary. Every crossing site $x \in X_B = \partial B \cap E(S')$ lies on exactly one child's boundary (the child that touches that segment of the parent boundary). We store a map

$$\phi^{\mathrm{out}}_{B \to i} : [4r]_B \supseteq \mathcal{P}^{\mathrm{out}}_{B,i} \to [4r]_{B_i},$$

mapping each parent slot on the outer boundary segment owned by child i to the corresponding child slot.

(2) Child-to-child maps on shared internal boundaries. The split of B into four children creates four shared internal boundary segments (two vertical half-segments and two horizontal half-segments). For each adjacent pair of children (i, j) sharing a segment, the same geometric crossing site appears in both interfaces, but with opposite boundary orientation. We store a bijection

$$\phi^{\mathrm{sh}}_{i \leftrightarrow j} : [4r]_{B_i} \supseteq \mathcal{P}^{\mathrm{sh}}_{i,j} \to [4r]_{B_j},$$

computed by matching sites by their normalized coordinate along the shared segment.

(3) Orientation conventions. Since pairings are defined using a clockwise order on ∂B (Appendix D.2), the shared-boundary correspondence must also record whether order is reversed. Our construction uses geometric coordinates and therefore is unambiguous.

All maps are constant-size objects (depending only on r) and can be computed once for each box.

E.2. Feasibility constraints

Let $\sigma = (a, \mathrm{mate}) \in \Omega(B)$ be the parent state and $\sigma_{B_i} = (a^{(i)}, \mathrm{mate}^{(i)}) \in \Omega(B_i)$ be child states. We say $\tau = (\sigma_{B_1}, \ldots, \sigma_{B_4})$ is feasible, i.e. $\tau \in \mathcal{F}_B(\sigma)$, iff all of the following hold.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 1 VERIFYTUPLE$(B, \sigma; \sigma_{B_1}, \ldots, \sigma_{B_4})$

Input: parent state $\sigma = (a, \mathrm{mate})$; child states $\sigma_{B_i} = (a^{(i)}, \mathrm{mate}^{(i)})$
Check C1: for each outer-boundary parent slot p owned by child i, verify $a_p = a^{(i)}_{\phi^{\mathrm{out}}_{B\to i}(p)}$; if not, **return Reject**.
Check C2: for each shared-boundary adjacency (i, j) and slot p, verify $a^{(i)}_p = a^{(j)}_{\phi^{\mathrm{sh}}_{i\leftrightarrow j}(p)}$; if not, **return Reject**.
Initialize an empty map $\mathrm{mate}^{\uparrow}$ on parent active slots.
for each active outer-boundary parent slot p not yet assigned in $\mathrm{mate}^{\uparrow}$ **do**
  Set $(i, q) \leftarrow$ LOCATECHILDSLOT(p) where $q = \phi^{\mathrm{out}}_{B\to i}(p)$.
  Trace q through the glued system until it exits on an outer-boundary slot:
    repeat: $q \leftarrow \mathrm{mate}^{(i)}(q)$;
    if q is shared-boundary, glue to neighbor: $(i, q) \leftarrow$ GLUE(i, q);
    until q lies on the outer boundary of B.
  Let p' be the parent slot corresponding to (i, q) via $(\phi^{\mathrm{out}}_{B\to i})^{-1}$.
  If the trace returned to p without reaching any outer boundary, **return Reject**. (C4)
  Set $\mathrm{mate}^{\uparrow}(p) = p'$ and $\mathrm{mate}^{\uparrow}(p') = p$.
end for
Check C3: verify $\mathrm{mate}^{\uparrow}(\cdot)$ equals the parent pairing $\mathrm{mate}(\cdot)$. If not, **return Reject**. Otherwise **return Accept**.

C1. Outer-boundary activation agreement. For every parent slot p that lies on the outer boundary of B and belongs to child i, the corresponding child slot $q = \phi^{\mathrm{out}}_{B\to i}(p)$ must satisfy

$$a_p \;=\; a^{(i)}_q.$$

(Intuitively, the parent decides whether the tour crosses ∂B at that site, and the child must realize the same decision on its portion of the boundary.)

C2. Shared-boundary activation agreement. For every adjacent pair (i, j) and every shared-boundary slot $p \in \mathcal{P}^{\mathrm{sh}}_{i,j}$, let $q = \phi^{\mathrm{sh}}_{i\leftrightarrow j}(p)$. Then

$$a^{(i)}_p \;=\; a^{(j)}_q.$$

Thus, crossings on a shared internal boundary are either present in both children or absent in both.

C3. Connectivity composition yields the parent pairing. Consider the following *gluing* operation. Each child state pairs its active slots into path endpoints inside that child. Each active shared-boundary slot p in child i is glued to the corresponding active slot q in child j, forming a continuation of the tour across the internal boundary.

After gluing all shared-boundary active sites, the remaining *exposed endpoints* are exactly the active outer-boundary slots of the parent. These exposed endpoints must be paired according to the parent map $\mathrm{mate}(\cdot)$.

C4. No internal closed component. The gluing procedure must not create a closed cycle component entirely within B (i.e., a component that does not touch the outer boundary of B). Such components are disallowed by our multipath-style legal family and correspond to premature subtours at this scale. (At the root box, the $p = 0$ entry corresponds to a single cycle.)

E.3. Exact verification via tracing

Because the interface size is $O(r)$, feasibility can be checked exactly by a constant-time tracing algorithm. We provide a deterministic procedure that (i) checks C1–C2, (ii) computes the induced parent pairing from children by tracing through mates and glued shared-boundary sites, and (iii) rejects if it encounters an internal cycle (C4).

Complexity. Each trace step either follows an intra-child mate or crosses a shared boundary once. There are at most $O(r)$ active interface slots per child, hence the total number of steps across all traces is $O(r^2)$, i.e. constant for fixed ε.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

E.4. A feasibility lemma: equivalence between VERIFYTUPLE and gluing semantics

We now formalize that the tuple verifier in Algorithm 1 is *exact*: it accepts if and only if the child states glue to the prescribed parent boundary state and do not form an internal closed component. This closes the loop between the structural legality guarantee (Lemma C.1) and the DP feasibility definition.

The gluing graph. Fix an internal box B with children $B_1, \ldots, B_4$ and child states $\sigma_{B_i} = (a^{(i)}, \mathrm{mate}^{(i)})$. Define a graph G_{glue} on *child-side slots* as follows. Its vertex set consists of all active child interface slots:

$$V := \{(i,p) : i \in \{1,2,3,4\},\ M_{B_i,p} = 1,\ a_p^{(i)} = 1\}.$$

There are two types of undirected edges in G_{glue}:

1. **Mate edges (inside children):** for each $(i,p) \in V$, add an edge between (i,p) and $(i, \mathrm{mate}^{(i)}(p))$.
2. **Glue edges (across shared boundaries):** if $(i,p) \in V$ lies on a shared boundary with a neighbor child j and $q = \phi^{\mathrm{sh}}_{i \leftrightarrow j}(p)$, add an edge between (i,p) and (j,q).

Under the boundary-agreement constraints (C2), glue edges pair up active shared-boundary slots consistently.

Let $U \subseteq V$ be the subset of vertices corresponding to *outer-boundary* active slots (those that map to active parent slots via ϕ^{out}). Every connected component of G_{glue} is then either: (i) a path whose endpoints are in U, or (ii) a cycle entirely contained in the interior (no vertex in U).

Induced parent pairing. Assuming there is no internal cycle component, each component is a path connecting two outer-boundary endpoints in U. This defines an induced pairing $\mathrm{mate}^{\uparrow}$ on the active parent slots: for an active parent slot p, trace from its corresponding child vertex in U along the unique path in G_{glue} to the other endpoint, and map that endpoint back to the corresponding parent slot p'; set $\mathrm{mate}^{\uparrow}(p) = p'$.

Lemma E.1 (Exactness of tuple verification)**.** *For a parent state $\sigma = (a, \mathrm{mate}) \in \Omega(B)$ and child states σ_{B_i}, Algorithm 1 returns **Accept** if and only if all of the following hold:*

1. *(**Activation agreement**) Child activations satisfy C1 and C2 (outer-boundary and shared-boundary agreement).*
2. *(**No internal closed component**) The gluing graph G_{glue} has no cycle component disjoint from U. Equivalently, every connected component intersects U in exactly two vertices.*
3. *(**Parent pairing match**) The induced pairing $\mathrm{mate}^{\uparrow}$ equals the parent pairing mate on all active parent slots.*

Proof. We prove both directions.

($\Rightarrow$) **If VERIFYTUPLE accepts, then (1)–(3) hold.** Algorithm 1 explicitly checks C1 and C2 before any tracing, hence (1) holds. Next, for each active parent slot p it traces alternately along child mate edges and glue edges until it exits at an outer-boundary slot; if a trace fails to exit (i.e., returns to the start without encountering the outer boundary), the algorithm rejects (line marked as C4). Therefore acceptance implies that no trace is trapped in a purely internal cycle, which is exactly condition (2). Finally, the algorithm constructs $\mathrm{mate}^{\uparrow}$ by tracing from each active parent slot p to the unique outer-boundary partner p' and then checks equality $\mathrm{mate}^{\uparrow} = \mathrm{mate}$. Thus acceptance implies (3).

($\Leftarrow$) **If (1)–(3) hold, then VERIFYTUPLE accepts.** Assume (1). Then all outer-boundary and shared-boundary activations are consistent, so G_{glue} is well-defined: every active shared-boundary vertex has exactly one glue edge to the matching vertex in the adjacent child. Also each active vertex has exactly one mate edge inside its child (since each child state is a perfect pairing on its active slots).

Now assume (2). Then G_{glue} has no internal cycle components. Hence starting from any outer-boundary vertex in U and following alternating mate/glue edges cannot get trapped in a cycle; it must reach the other endpoint in the same path component, which is also in U. Therefore every trace performed by VERIFYTUPLE terminates at an outer-boundary slot and never triggers the cycle-rejection case.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

The traces computed by VERIFYTUPLE are exactly the path traversals in G_{glue}, so the partner map it constructs is precisely $\text{mate}^{\uparrow}$. By (3), $\text{mate}^{\uparrow}$ matches the parent pairing mate, so the final check passes and the algorithm accepts.

This completes the proof. □

Implication for DP correctness. Lemma E.1 shows that the verifier enforces exactly the multipath-style composition semantics used in Arora's portal DP (Arora, 1998), but with crossing sites defined by $X_B = \partial B \cap E(S')$. Combined with Lemma C.1, this ensures that optimizing over feasible tuples indeed captures an $(1 + O(\varepsilon))$-optimal tour within the legal family $\mathcal{S}_{legal}$.

E.5. Relation to Arora's multipath enumeration

In Arora's DP, feasibility is enforced by enumerating all ways a multipath can cross child boundaries, including the order and pairing of internal portals (Arora, 1998). Our feasibility definition captures the same semantics in a spanner-based interface: the enumeration (in exact DP) ranges over child states whose shared-boundary activations match and whose connectivity composition induces the parent state. The key simplification enabled by the r-light patched spanner is that the interface points are the *actual* intersections $X_B = \partial B \cap E(S')$ (size $O(r)$) rather than $m = O(\log L/\varepsilon)$ uniform portals, and therefore $\Omega(B)$ depends only on $r(\varepsilon)$.

F. Parsing continuous boundary states

This appendix specifies the deterministic PARSE procedure that projects a continuous boundary state $\tilde{\sigma}$ (decoder output) onto a discrete DP state $\sigma \in \Omega(B)$, and its top-K extension.

F.1. Continuous relaxation: a concrete parameterization

Fix a box B and its $4r$ interface slots with mask M_B (Appendix D.1). We represent a continuous boundary state $\tilde{\sigma}$ by

$$\tilde{\sigma} \equiv \big(\alpha,\ W\big),$$

where:

- $\alpha \in \mathbb{R}^{4r}$ are *activation logits*. Intuitively, $\sigma(\alpha_p)$ is the model's preference that slot p is active, and we always mask empty slots by setting $\alpha_p = -\infty$ whenever $M_{B,p} = 0$.
- $W \in \mathbb{R}^{4r \times 4r}$ are *pairing logits*, where W_{pq} scores the preference to pair slot p with slot q within the same box. We only consult entries with $p \neq q$ and with both slots present.

This choice is intentionally simple: it is fixed-dimensional (depends only on r), and it supports a deterministic projection onto a noncrossing pairing by a small DP (since $4r$ is constant).

F.2. Hard constraints induced by the parent state and shared boundaries

In the merge for an internal box B with children $(B_1, \ldots, B_4)$, PARSE is invoked on the decoded continuous tuple $\tilde{\tau}(\sigma) = (\tilde{\sigma}_{B_1}, \ldots, \tilde{\sigma}_{B_4})$. Although each child has its own local interface, there are *hard* agreement constraints:

(i) Outer-boundary constraints from the parent state. A subset of each child boundary lies on the parent boundary ∂B. For every child slot p that corresponds to a crossing site on ∂B, the activation decision is fixed by the parent state $\sigma \in \Omega(B)$: if the parent requires that crossing to be active/inactive, we overwrite the child logit α_p with $+\infty$ / $-\infty$, respectively. This ensures that the parsed child states agree with σ on outer-boundary crossings.

(ii) Shared-boundary agreement between adjacent children. If two adjacent children share an internal boundary segment, the crossing sites along that segment are the same geometric points seen from either side. We precompute a slot correspondence map between the two children; for each matched slot pair (p, q), we reconcile activation logits by setting

$$\alpha_p^{(1)} \leftarrow \tfrac{1}{2}\big(\alpha_p^{(1)} + \alpha_q^{(2)}\big), \qquad \alpha_q^{(2)} \leftarrow \alpha_p^{(1)},$$

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

(and analogously if using probabilities). Pairing logits remain intra-child and are not tied across children. After this reconciliation, the two children will select the *same* active set on their shared boundary.

These reconciliations are purely local and deterministic; they do not replace the DP feasibility check $\tau(\sigma) \in \mathcal{F}_B(\sigma)$, which is still enforced exactly after parsing.

F.3. Deterministic projection to a discrete state

We now describe how PARSE maps a single child's continuous state $\tilde{\sigma} = (\alpha, W)$ to a discrete state $\sigma = (a, \mathrm{mate}) \in \Omega(B)$.

Step 1: choose an even active set. Let $S := \{p \in [4r] : M_{B,p} = 1\}$ be present slots. We first compute activation scores $s_p := \alpha_p$ for $p \in S$ and sort them descending. We then choose the active set as the best-scoring *even-cardinality* subset:

$$A^\star \in \arg \max_{A \subseteq S,\ |A| \text{ even}} \sum_{p \in A} s_p - \lambda |A|,$$

where $\lambda \geq 0$ is a fixed penalty that discourages overly large interfaces (in practice, λ is a small constant). Since $|S| \leq 4r$ is constant, this maximization can be done by trying all even sizes $|A| = 0, 2, 4, \ldots, 4r$ and taking the best prefix under the sorted scores. (Any other deterministic even-selection rule is also valid.)

Step 2: maximum-weight noncrossing perfect matching. Let the chosen active slots be $A^\star = \{p_1, \ldots, p_{2k}\}$ sorted by the clockwise order of their underlying geometric sites on ∂B. Define an edge weight between active indices $i < j$ by

$$w(i, j) := W_{p_i p_j} + W_{p_j p_i}.$$

We find the best noncrossing pairing by a small dynamic program over intervals:

$$\mathrm{DP}[i, j] = \max_{t \in \{i+1, i+3, \ldots, j\}} \Big(\mathrm{DP}[i+1, t-1] + \mathrm{DP}[t+1, j] + w(i, t) \Big), \tag{13}$$

with base cases $\mathrm{DP}[i, i-1] = 0$ and $\mathrm{DP}[i, i] = -\infty$. This is the standard "RNA folding" DP and runs in $O(k^3)$ time, which is constant since $k \leq 2r$. We recover the optimal pairing by traceback and set $\mathrm{mate}(p_i) = p_{\pi(i)}$.

Step 3: output. We output the discrete state $\sigma = (a, \mathrm{mate})$ where $a_p = \mathbb{1}[p \in A^\star]$ and mate is the partner map from Step 2 (inactive/empty slots map to $\perp$). By construction, σ satisfies the mask constraint, even parity, and noncrossing pairing requirements.

F.4. Top-K parsing (optional)

To reduce fallback frequency without changing verifiability, PARSE can emit a small top-K list of candidate states (or candidate child tuples) ranked by a *parser score*. A simple instantiation is:

- Enumerate a small set of activation candidates by flipping the m most uncertain logits (e.g., those with smallest $|\alpha_p|$), keeping only even-cardinality sets.
- For each activation set, solve the maximum-weight noncrossing matching by (13).
- Rank candidates by $\sum_{p \in A} \alpha_p + \sum_{(i,t) \in \pi} w(i, t)$ and return the top K.

Because $4r$ is constant, even an exhaustive enumeration over a small Hamming ball around $A^\star$ is cheap and keeps the parser deterministic.

F.5. Putting it together: PARSE for a 4-tuple

Algorithm 2 summarizes the tuple-level parser used in the merge. It first enforces the hard agreement constraints (outer boundary from the parent state, and shared boundaries between adjacent children), then parses each child independently using the noncrossing-matching projection above.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 2 PARSETUPLE$(\sigma, \tilde{\sigma}_{B_1}, \ldots, \tilde{\sigma}_{B_4})$

Input: parent state $\sigma \in \Omega(B)$; decoded continuous child states $\tilde{\sigma}_{B_i} = (\alpha^{(i)}, W^{(i)})$
for each child i **do**
  mask empty slots in $\alpha^{(i)}$ using M_{B_i}
  overwrite outer-boundary slots in $\alpha^{(i)}$ to match σ (hard constraint)
end for
for each adjacent child pair (i, j) sharing an internal boundary **do**
  reconcile activation logits on corresponding shared-boundary slots by averaging (hard agreement)
end for
for each child $i = 1, \ldots, 4$ **do**
  $\sigma_{B_i} \leftarrow$ PARSEONE$(\alpha^{(i)}, W^{(i)})$ (even selection + noncrossing matching)
end for
Output: tuple $\tau(\sigma) = (\sigma_{B_1}, \ldots, \sigma_{B_4})$

Verification and fallback. After PARSETUPLE, we still enforce DP semantics exactly: accept the tuple only if it lies in $\mathcal{F}_B(\sigma)$ and compute the merge value from the child tables. Otherwise, we trigger the entry-level fallback (exact enumeration for this (B, σ) entry).

G. Training details

Training data and teacher tours. We train the shared encoder–decoder only on synthetic Euclidean TSP instances with $n = 50$. For each instance, we compute a high-quality reference tour using multi-start LKH-3 (multiple random seeds) and retain the best tour found; we use this tour as a pseudo-optimal teacher label. For a subset with $n \leq 50$, we additionally verify optimality with an exact solver, and empirically find that the LKH-3 teacher is optimal on those samples.

Supervision on the fixed local interface. Training targets are defined on the same fixed-dimensional local interface used by the top-down pass. When a box B is visited under a conditioning signal produced by the current model, the decoder predicts four continuous child boundary states $\tilde{\sigma}^{\text{child}}_{B \to B_i} \in \mathbb{R}^{d_{\text{child}}}$ for $i \in \{1, 2, 3, 4\}$. Optionally, we also predict auxiliary continuous boundary-state heads $\tilde{\sigma}^{\text{iface}}_B \in \mathbb{R}^{d_{\text{iface}}}$ and $\tilde{\sigma}^{\text{cross}}_B \in \mathbb{R}^{d_{\text{cross}}}$. Given the teacher tour, we derive binary labels $y^{\text{child}}_{B \to B_i} \in \{0, 1\}^{d_{\text{child}}}$ (and optionally $y^{\text{iface}}_B, y^{\text{cross}}_B$), where each coordinate indicates whether the corresponding local boundary/interface decision is used by the teacher tour.

Because not all coordinates are well-defined for every realized visit (e.g., absent coordinates or pruned interfaces), we construct a matching binary mask for each head, $m \in \{0, 1\}^d$, and supervise only active coordinates.

Masked binary cross-entropy objective. For a coordinate vector $\tilde{\sigma} \in \mathbb{R}^d$ with labels $y \in \{0, 1\}^d$ and mask $m \in \{0, 1\}^d$, define

$$\text{MBCE}(\tilde{\sigma}, y, m) := \begin{cases} \frac{1}{\|m\|_1} \sum_{j:\, m_j = 1} \Big[-y_j \log g(\tilde{\sigma}_j) - (1 - y_j) \log(1 - g(\tilde{\sigma}_j)) \Big], & \|m\|_1 > 0, \\ 0, & \|m\|_1 = 0, \end{cases}$$

where $g(\cdot)$ is the elementwise sigmoid. For a visited box B, the per-box loss is

$$\mathcal{L}(B) = w_{\text{child}} \sum_{i=1}^{4} \text{MBCE}\big(\tilde{\sigma}^{\text{child}}_{B \to B_i}, y^{\text{child}}_{B \to B_i}, m^{\text{child}}_{B \to B_i}\big) + w_{\text{iface}} \, \text{MBCE}\big(\tilde{\sigma}^{\text{iface}}_B, y^{\text{iface}}_B, m^{\text{iface}}_B\big) + w_{\text{cross}} \, \text{MBCE}(\tilde{\sigma}^{\text{cross}}_B, y^{\text{cross}}_B, m^{\text{cross}}_B) \,,$$

dropping the auxiliary terms when the corresponding heads are disabled. The total loss sums $\mathcal{L}(B)$ over all boxes visited by the top-down schedule across all training instances.

Free-running conditioning. We train under free-running conditioning to match the inference distribution: during training we execute the same top-down schedule as at test time, so each box B is visited once under the conditioning signal produced by the current model (rather than teacher forcing). The masks ensure we only supervise interface decisions that are well-defined under the realized visit, while the teacher tour supplies consistent pseudo labels on the underlying instance.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.